WACHEMO UNIVERSITY
COLLEGE OF ENGINEERING AND TECHNOLOGY
SCHOOL OF COMPUTING AND INFORMATION
DEPARTMENT OF INFORMATION TECHNOLOGY

Group 6 OS python code

Name
ID
1, Henok Brihanu……176534
2,Zerihun Keto……17D5880

1, First-Come, First-Served Scheduling

- Simplest scheduling algorithm that scheduling according to arrival times of process.
- Jobs are execute on first come, first served basis.

Easy to understand and implement.

With python code

```
def fcfs_scheduling(process_list):
    process_list.sort()

    t = 0
    total_wt = 0
    total_tt = 0
    n = len(process_list)

    print(f"{'PID':<6} | {'AT':<4} | {'BT':<4} | {'CT':<4} | {'TT':<4} | {'WT':<4}")
    print("-" * 40)

    for at, bt, pid in process_list:

        if at > t:
            t = at

        start_time = t
        t += bt

        ct = t            # Completion Time
        tt = ct - at      # Turnaround Time
        wt = tt - bt      # Waiting Time

        total_wt += wt
        total_tt += tt

        print(f"{pid:<6} | {at:<4} | {bt:<4} | {ct:<4} | {tt:<4} | {wt:<4}")
```

```
    awt = total_wt / n
    att = total_tt / n
```

```
    print("-" * 40)
    print(f"Average Waiting Time (AWT): {awt:.2f}")
    print(f"Average Turnaround Time (ATT): {att:.2f}")
```

```
if __name__ == "__main__":
    # Format: [Arrival Time, Burst Time, PID]
    my_processes = [[0, 24, "P1"], [0, 3, "P2"], [0, 3, "P3"]]
    fcfs_scheduling(my_processes)
```

Output

PID	AT	BT	CT	TT	WT
P2	0	3	3	3	0
P3	0	3	6	6	3
P1	0	24	30	30	6

Average Waiting Time (AWT): 3.00
Average Turnaround Time (ATT): 13.00

2, Shortest-Job-First Scheduling

- This algorithm associate with each process the length of the process next CPU burst.
- Process which have shortest burst time.

```
def sjf_scheduling(processes_input):
    # processes_input = [[pid, arrival_time, burst_time]]
    n = len(processes_input)

    processes = [list(p) for p in processes_input]
    processes.sort(key=lambda x: x[1])

    completed = [False] * n
    completion_time = [0] * n
    waiting_time = [0] * n
    turnaround_time = [0] * n

    current_time = 0
    completed_count = 0
    gantt = []

    while completed_count < n:

        available_processes = []
        for i in range(n):
            if processes[i][1] <= current_time and not completed[i]:
                available_processes.append(i)

        if not available_processes:

            current_time += 1
            continue


        idx = min(available_processes, key=lambda i: processes[i][2])


        gantt.append(processes[idx][0])
        current_time += processes[idx][2]
        completion_time[idx] = current_time
        turnaround_time[idx] = completion_time[idx] - processes[idx][1]
        waiting_time[idx] = turnaround_time[idx] - processes[idx][2]

        completed[idx] = True
        completed_count += 1
```

```
    print(f"Gantt Chart: {' -> '.join(gantt)}")
    print("-" * 55)
    print("PID\tArrival\tBurst\tCT\tTAT\tWT")
```

```
    total_tat = 0
    total_wt = 0
    for i in range(n):
        total_tat += turnaround_time[i]
        total_wt += waiting_time[i]
        print(f"{processes[i][0]}\t{processes[i][1]}\t{processes[i][2]}\t{completion_time[i]}\
t{turnaround_time[i]}\t{waiting_time[i]}")
```

```
    print("-" * 55)
    print(f"Average Turnaround Time: {total_tat / n:.2f}")
    print(f"Average Waiting Time: {total_wt / n:.2f}")
```

```
proc = [
    ["P1", 0, 6],
    ["P2", 0, 8],
    ["P3", 0, 7],
    ["P4", 3, 3]
]
sjf_scheduling(proc)
```

OUTPUT:

Gantt Chart: P4 -> P1 -> P3 -> P2

PID	Arrival	Burst	CT	TAT	WT
P1	0	6	9	9	3
P2	0	8	24	24	16
P3	0	7	16	16	9
P4	0	3	3	3	0

Average Turnaround Time: 13.00
Average Waiting Time: 7.00

3, Priority CPU scheduling (Non pre-emptive)

```
def priority(process_list):
    gantt = []
    t = 0
    completed = {}
    n = len(process_list)

    # Keep running until all processes are handled
    while process_list:
        available = []
        for p in process_list:
            arrival_time = p[3]
            if arrival_time <= t:
                available.append(p)

        if not available:
            gantt.append("Idle")
            t += 1
            continue
        else:
            # Sort by Priority (index 0) - Lower number usually means higher priority
            available.sort(key=lambda x: x[0])

            process = available[0]
```

```
            process_list.remove(process)

            priority_val = process[0]
            pid = process[1]
            burst_time = process[2]
            arrival_time = process[3]

            gantt.append(pid)
            t += burst_time  # Move time forward by burst time

            ct = t
            tt = ct - arrival_time
            wt = tt - burst_time
            completed[pid] = [ct, tt, wt]

    # Print Results
    print("\nGantt Chart:", " -> ".join(gantt))
    print("-" * 50)
    print("PID\tCT\tTAT\tWT")

    total_wt = 0
    total_tat = 0

    for pid, metrics in completed.items():
        print(f"{pid}\t{metrics[0]}\t{metrics[1]}\t{metrics[2]}")
        total_tat += metrics[1]
        total_wt += metrics[2]

    avg_tat = total_tat / n
    avg_wt = total_wt / n

    print("-" * 50)
    print(f"Average Turnaround Time: {avg_tat:.2f}")
    print(f"Average Waiting Time:    {avg_wt:.2f}")
    print("-" * 50)
```

```
if __name__ == "__main__":
    # Format: [Priority, PID, Burst, Arrival]
    my_processes = [
        [3, "P1", 10, 0],
        [1, "P2", 1, 0],
        [4, "P3", 2, 0],
        [5, "P4", 1, 0],
        [2, "P5", 5, 0]
    ]
    priority(my_processes)
```

OUTPUT:

Gantt Chart: P2 -> P5 -> P1 -> P3 -> P4

PID	CT	TAT	WT
P2	1	1	0
P5	6	6	1
P1	16	16	6
P3	18	18	16
P4	19	19	18

Average Waiting Time: 8.20

Average Turn around time:12

4, Round-Robin Scheduling

Scheduling algorithm is designed special for timesharing system.

The python code

```
def round_robin(process_list, time_quanta):
    t = 0
    gantt = []
    completed = {}
    # Use a real queue for Round Robin
    queue = []
    process_list.sort(key=lambda x: x[1]) # Sort by Arrival Time

    # Store original burst times for calculation
    burst_times = {p[0]: p[2] for p in process_list}

    while process_list or queue:
        # Move arrived processes to the queue
        while process_list and process_list[0][1] <= t:
            queue.append(process_list.pop(0))

        if not queue:
            gantt.append("Idle")
            t += 1
            continue

        process = queue.pop(0)
        pid, arrival, rem_burst = process[0], process[1], process[2]

        gantt.append(pid)

        if rem_burst <= time_quanta:
            t += rem_burst
            ct = t
            tt = ct - arrival
            wt = tt - burst_times[pid]
            completed[pid] = [ct, tt, wt]
        else:
            t += time_quanta
            process[2] -= time_quanta
            # Check for new arrivals during the time slice before re-adding
            while process_list and process_list[0][1] <= t:
                queue.append(process_list.pop(0))
            queue.append(process)

    print("Gantt Chart:", " -> ".join(gantt))
    print("Results (PID: [CT, TAT, WT]):", completed)
```

```
if __name__ == "__main__":
    # Format: [PID, Arrival, Burst]
    my_processes = [["p1", 2, 6],
 ["p2", 5, 2],
["p3", 1, 8],
["p4", 0, 3]]
    round_robin(my_processes, 2)
```

OUTPUT:

Gantt Chart:P4 -> P3 -> P1 ->P4 ->P3 -> P2 -> P1 -> P3 -> P1 -> P3

Results	PID:	CT	TAT,	WT
	P4:	7	7	4
	P2:	11	6	4
	P1:	17	15	9
	P3:	19	18	10

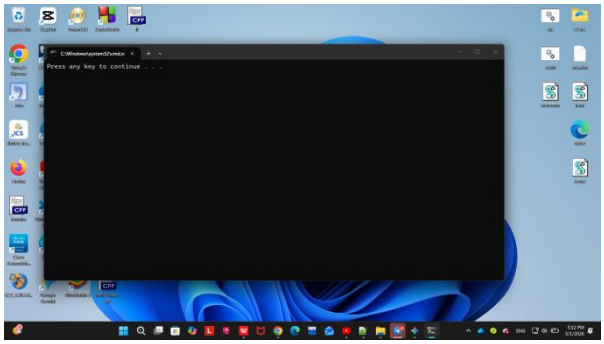
SECOND PROJECT

Ex1 Use the trick to create a batch which will speak a welcome message to you whenever you log into your machine

Code#1

```
@echo off
set text="Hello, Welcome back to IT students class this message of for all students the final exam will start in next week and ready all students ."
echo set speech = Wscript.CreateObject("SAPI.spVoice") > "temp.vbs"
echo speech.speak %text% >> "temp.vbs"
start temp.vbs
pause
del temp.vbs
```

Output:



Ex2 Create a batch file that can hide or unhide files or folder she your desktop or laptop

Code#2

```
@ECHO OFF
title Hide or Unhide Folder
set /p choice="Type H to hide or U to unhide: "
if %choice%==H goto HIDE
```

```
if %choice%==U goto UNHIDE
:HIDE
attrib +h +s "private"
echo Folder hidden.
pause
exit

:UNHIDE
attrib -h -s "private"
echo Folder unhidden.
pause
Exit
```

Output: